

# Docker and Kubernetes



# Lesson Objectives

Session 1 – Getting started with Docker

Session 2 – Building Docker images

Session 3 – Docker Networking

Session 4 – Docker storage

Session 5 – Making a real project with Docker and NodeJs

Session 6 – Docker Swarm

Session 7-10 – Kubernetes

Session 11 – Final test

## Session 10

# Kubernetes (Cont.)

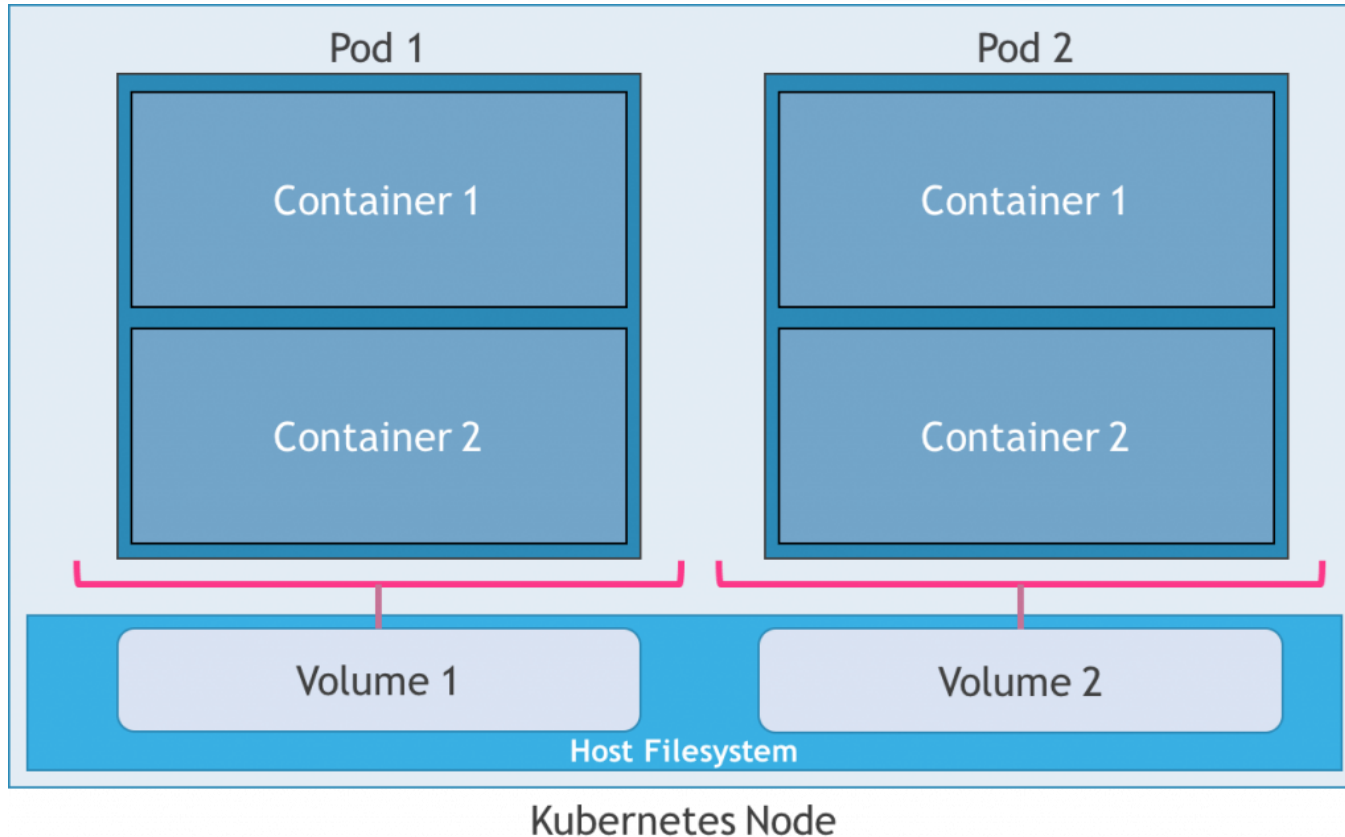
# Agenda

- Assignment Review & Guides
- Kubernetes volume management
- Configmap and secret

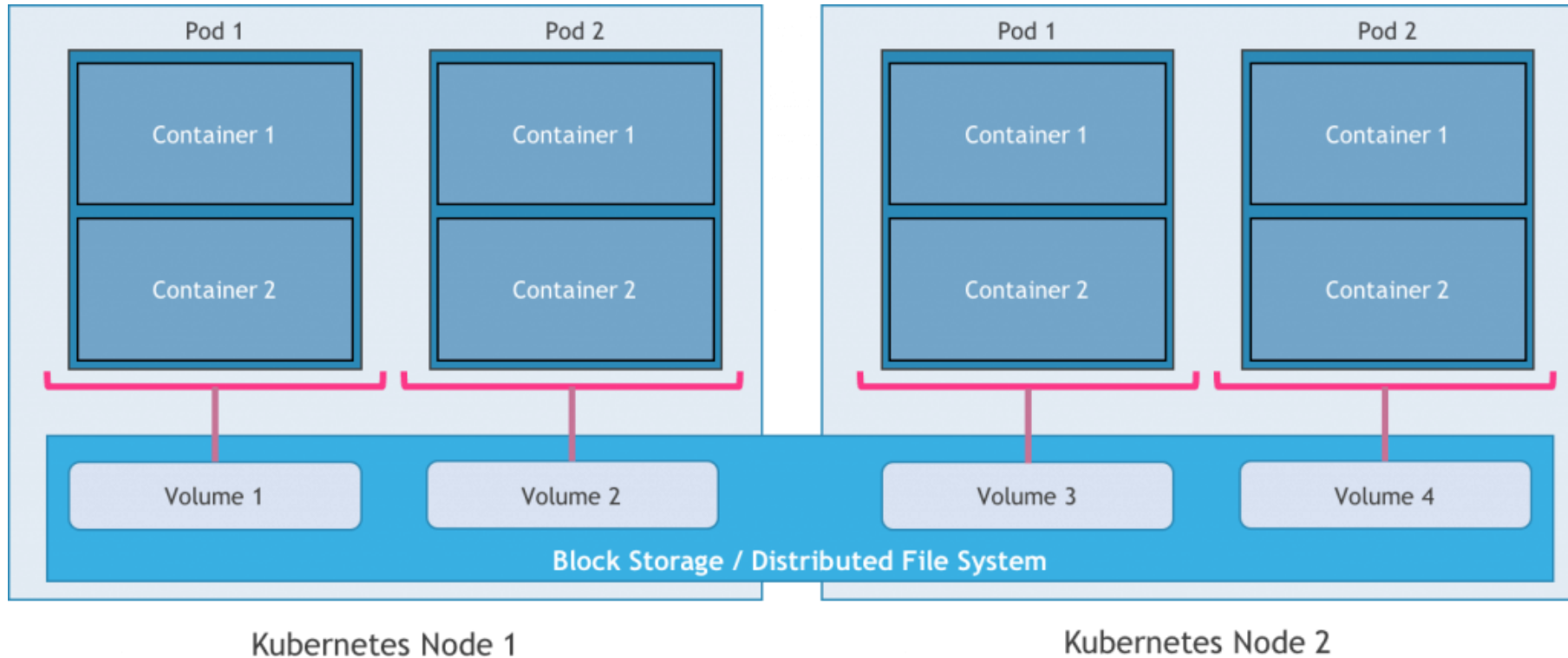
# STORAGE

- Pod are ephemeral and stateless
- Volumes bring persistence to Pods
- Kubernetes volumes are similar to Docker volumes, but managed differently
- All containers in a Pod can access the volume
- Volumes are associated with the lifecycle of Pod
- Directories in the host are exposed as volumes
- Volumes may be based on a variety of storage backend
- A pod can have one or more types of volumes attached to it

# Pod and Volumes



# Pod and Volumes





- **Host-based**
  - ❖ emptyDir
  - ❖ hostPath
- **Block Storage**
  - ❖ awsElasticBlockStore
  - ❖ azureDisk
  - ❖ gcePersistentDisk
  - ❖ vsphereVolume
  - ❖ ...
- **Distributed File System**
  - ❖ NFS
  - ❖ Ceph
  - ❖ Gluster
  - ❖ Amazon EFS
  - ❖ Azure File System
  - ❖ ...
- **Other**
  - ❖ Flocker
  - ❖ iScsi

If the EBS volume is partitioned, you can supply the optional field **partition**: "**<partition number>**" to specify which partition to mount on

```
apiVersion: v1
kind: Pod
metadata:
  name: test-eks
spec:
  containers:
  - image: k8s.gcr.io/test-webserver
    name: test-container
    volumeMounts:
    - mountPath: /test-eks
      name: test-volume
  volumes:
  - name: test-volume
    # This AWS EBS volume must already exist.
    awsElasticBlockStore:
      volumeID: "<volume id>"
      fsType: ext4
```

# Azure Disk configuration example

```
kind: Pod
apiVersion: v1
metadata:
  name: pod-uses-managed-ssd-5g
  labels:
    name: storage
spec:
  containers:
  - image: nginx
    name: az-c-01
    command:
    - /bin/sh
    - -c
    - while true; do echo $(date) >> /mnt/managed/outfile; sleep 1; done
  volumeMounts:
  - name: managed01
    mountPath: /mnt/managed
  volumes:
  - name: managed01
    azureDisk:
      kind: Managed
      diskName: myDisk
      diskURI: /subscriptions/<subscriptionID>/resourceGroups/<resourceGroup>/providers/Microsoft.Compute/disks/<diskName>
```

An emptyDir volume is first created when a Pod is assigned to a node, and exists as long as that Pod is running on that node. As the name says, the emptyDir volume is initially empty

```
apiVersion: v1
kind: Pod
metadata:
  name: test-pd
spec:
  containers:
    - image: k8s.gcr.io/test-webserver
      name: test-container
      volumeMounts:
        - mountPath: /cache
          name: cache-volume
  volumes:
    - name: cache-volume
      emptyDir: {}
```

# hostPath configuration example

A hostPath volume mounts a file or directory from the host node's filesystem into your Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: test-pd
spec:
  containers:
    - image: k8s.gcr.io/test-webserver
      name: test-container
      volumeMounts:
        - mountPath: /test-pd
          name: test-volume
  volumes:
    - name: test-volume
      hostPath:
        # directory location on host
        path: /data
        # this field is optional
        type: Directory
```

A PersistentVolume (PV) is a piece of storage in the cluster that has been provisioned by an administrator or dynamically provisioned using Storage Classes.

It is a resource in the cluster just like a node is a cluster resource. PVs are volume plugins like Volumes, but have a lifecycle independent of any individual Pod that uses the PV

- A **PersistentVolume** (PV) represents a storage resource.
- PVs are a **cluster wide resource** linked to a backing storage provider: NFS, EBS, GCEPersistentDisk, etc.
- Generally provisioned by an administrator.
- Their lifecycle is handled independently from a pod
- **CANNOT** be attached to a Pod directly. Relies on a **PersistentVolumeClaim** (PVC).
- A PVC is a **namespaced** request for storage.

A PersistentVolumeClaim (PVC) is a request for storage by a user.

It is similar to a Pod. Pods consume node resources and PVCs consume PV resources. Pods can request specific levels of resources (CPU and Memory).

Claims can request specific size and access modes (e.g., they can be mounted ReadWriteOnce, ReadOnlyMany or ReadWriteMany).



There are two ways PVs may be provisioned: statically or dynamically

## ❖ Static

A cluster administrator creates a number of PVs. They carry the details of the real storage, which is available for use by cluster users. They exist in the Kubernetes API and are available for consumption.

## ❖ Dynamic

When none of the static PVs the administrator created match a user's PersistentVolumeClaim, the cluster may try to dynamically provision a volume specially for the PVC. This provisioning is based on StorageClasses

Each PV contains a spec and status, which is the specification and status of the volume

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0003
spec:
  capacity:
    storage: 5Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Recycle
  storageClassName: slow
  mountOptions:
    - hard
    - nfsvers=4.1
  nfs:
    path: /tmp
    server: 172.17.0.2
```

Kubernetes supports two volumeModes of PersistentVolumes: **Filesystem** and **Block**

A volume with volumeMode: **Filesystem** is mounted into Pods into a directory. If the volume is backed by a block device and the device is empty, Kubernetes creates a filesystem on the device before mounting it for the first time.

You can set the value of volumeMode to **Block** to use a volume as a raw block device. Such volume is presented into a Pod as a block device, without any filesystem on it. This mode is useful to provide a Pod the fastest possible way to access a volume, without any filesystem layer between the Pod and the volume

The access modes are:

- ❖ `ReadWriteOnce` -- the volume can be mounted as read-write by a single node
- ❖ `ReadOnlyMany` -- the volume can be mounted read-only by many nodes
- ❖ `ReadWriteMany` -- the volume can be mounted as read-write by many nodes
- ❖ `ReadWriteOncePod` -- the volume can be mounted as read-write by a single Pod. This is only supported for CSI volumes and Kubernetes version 1.22+.

In Kubernetes, a VolumeSnapshot represents a snapshot of a volume on a storage system

Similar to how API resources PersistentVolume and PersistentVolumeClaim are used to provision volumes for users and administrators, VolumeSnapshotContent and VolumeSnapshot API resources are provided to create volume snapshots for users and administrators

# Volume Snapshots configuration example

Each VolumeSnapshot contains a spec and a status

persistentVolumeClaimName is the name of the PersistentVolumeClaim data source for the snapshot. This field is required for dynamically provisioning a snapshot

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: new-snapshot-test
spec:
  volumeSnapshotClassName: csi-hostpath-snapclass
  source:
    persistentVolumeClaimName: pvc-test
```

volumeHandle is the unique identifier of the volume created on the storage backend and returned by the CSI driver during the volume creation

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotContent
metadata:
  name: snapcontent-72d9a349-aacd-42d2-a240-d775650d2455
spec:
  deletionPolicy: Delete
  driver: hostpath.csi.k8s.io
  source:
    volumeHandle: ee0cfb94-f8d4-11e9-b2d8-0242ac110002
  volumeSnapshotClassName: csi-hostpath-snapclass
  volumeSnapshotRef:
    name: new-snapshot-test
    namespace: default
    uid: 72d9a349-aacd-42d2-a240-d775650d2455
```

- A StorageClass provides a way for administrators to describe the "classes" of storage they offer.
- Satisfies a set of requirements instead of mapping to a storage resource directly.
- Ensures that an application's claim for storage is portable across numerous backends or providers.
- Different classes might map to quality-of-service levels, or to backup policies, or to arbitrary policies determined by the cluster administrators



- Each StorageClass contains the fields provisioner, parameters, and reclaimPolicy, which are used when a PersistentVolume belonging to the class needs to be dynamically provisioned.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: standard
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp2
reclaimPolicy: Retain
allowVolumeExpansion: true
mountOptions:
  - debug
volumeBindingMode: Immediate
```

# The StorageClass Provisioner

| Volume Plugin        | Internal Provisioner | Config Example                   |
|----------------------|----------------------|----------------------------------|
| AWSElasticBlockStore | ✓                    | <a href="#">AWS EBS</a>          |
| AzureFile            | ✓                    | <a href="#">Azure File</a>       |
| AzureDisk            | ✓                    | <a href="#">Azure Disk</a>       |
| CephFS               | -                    | -                                |
| Cinder               | ✓                    | <a href="#">OpenStack Cinder</a> |
| FC                   | -                    | -                                |
| FlexVolume           | -                    | -                                |
| Flocker              | ✓                    | -                                |

# The StorageClass Provisioner

|                   |   |                                 |
|-------------------|---|---------------------------------|
| GCEPersistentDisk | ✓ | <a href="#">GCE PD</a>          |
| Glusterfs         | ✓ | <a href="#">Glusterfs</a>       |
| iSCSI             | - | -                               |
| Quobyte           | ✓ | <a href="#">Quobyte</a>         |
| NFS               | - | <a href="#">NFS</a>             |
| RBD               | ✓ | <a href="#">Ceph RBD</a>        |
| VsphereVolume     | ✓ | <a href="#">vSphere</a>         |
| PortworxVolume    | ✓ | <a href="#">Portworx Volume</a> |
| ScaleIO           | ✓ | <a href="#">ScaleIO</a>         |
| StorageOS         | ✓ | <a href="#">StorageOS</a>       |
| Local             | - | <a href="#">Local</a>           |

# CONFIGURATION

- An API object used to store non-confidential data in key-value pairs
- Externalized data stored within kubernetes
- Can be referenced through several different means:
  - ❑ environment variable
  - ❑ a command line argument (via environment variable)
  - ❑ injected as a configuration file into a volume mount
- Can be created from a manifest, literals, directories, or files directly.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: manifest-example
data:
  state: Minnesota
  city: Minneapolis
  content: |
    Look at this,
    its multiline!
```

- Functionally identical to a ConfigMap.
- Stored as base64 encoded content.
- Encrypted at rest within etcd (if configured!).
- Ideal for username/passwords, certificates or other sensitive information that should not be stored in a container.
- Can be created from a manifest, literals, directories, or from files directly.

- **type**: There are three different types of secrets within Kubernetes:
  - **docker-registry** - credentials used to authenticate to a container registry
  - **generic/Opaque** - literal values from different sources
  - **tls** - a certificate based secret
- **data**: Contains key-value pairs of base64 encoded content.

```
apiVersion: v1
kind: Secret
metadata:
  name: manifest-secret
type: Opaque
data:
  username: ZXhhbXBsZQ==
  password: bXlwYXNzd29yZA==
```

# LAB

